

VAHAN VEHICLE REGISTRATION DATA ANALYSIS

Data Analysis Report

Using Python (Pandas), PostgreSQL / SQL and Microsoft Power BI

Vehicle Registration, Fleet Composition and Emission-Profile Analysis

Prepared as a data analytics and visualization project

1. Executive Summary

This project analyzes a Vahan vehicle-registration dataset to understand the composition, geographical distribution, registration trends, fuel profile, vehicle age, engine displacement and Bharat Stage emission-norm profile of registered vehicles. The analysis was designed as a data analytics workflow rather than as a simple descriptive dashboard: the raw data was cleaned and enriched in Python using Pandas, analyzed and stored in PostgreSQL for SQL-based querying, and finally modeled and visualized in Power BI through interactive and dynamic dashboards.

The final analytical dataset contains 3,700 vehicle-registration records in the working sample. The dashboard shows 1,648 two-wheelers, 1,160 four-wheelers and 968 vehicles classified as EVs after the cleaning and fuel-type standardization process. For vehicles with an applicable Bharat Stage classification, the current dataset contains 2,277 BS6, 451 BS4 and 4 BS3 records. The remaining 968 records correspond to EVs whose Bharat Stage classification is treated as Not Applicable rather than forcing them into BS3/BS4/BS6.

2. Project Objectives

1. Understand the structure and quality of the Vahan vehicle-registration data.
2. Clean inconsistent, missing, duplicated and incorrectly typed fields before analysis.
3. Standardize vehicle, fuel, state and RTO-related attributes so that aggregation is reliable.
4. Analyze registration trends over time and the distribution of vehicles across categories, fuels, manufacturers and states.
5. Profile the fleet using emission norms, vehicle age and engine displacement.
6. Separate pure EVs from ICE emission-norm analysis because BS3/BS4/BS6 classifications are not applicable to pure battery-electric vehicles.
7. Use SQL to perform reproducible aggregations and database-level analysis after loading the cleaned Pandas DataFrame into PostgreSQL.
8. Build a dynamic Power BI dashboard that allows users to filter the analysis by year, state, fuel type, emission norm and other dimensions.
9. Identify segments that can be prioritized for future, activity-based vehicle-emission estimation and reduction studies.

3. Dataset and Analytical Scope

The working dataset is a Vahan vehicle-registration dataset. Each row represents a vehicle-registration record in the supplied analytical sample. The project uses registration attributes together with vehicle characteristics and emission-related classifications.

Field / Group	Purpose in Analysis
Registration_Number	Vehicle-level identifier used for distinct vehicle counts.
Registration_Date	Temporal analysis and date-derived fields.
Registration_Year	Annual registration trends and vehicle-age calculation.
State / State_Clean	Geographical analysis and state ranking.
RTO_Office / RTO_State_Code	RTO-level and state-code analysis.
Vehicle_Category	2W, 4W and other vehicle categories.
Vehicle_Sub_Type	Detailed body/type profile such as scooter, motorcycle, hatchback, etc.

Manufacturer_Brand	Manufacturer ranking and market composition.
Fuel_Type / Fuel_Type_Clean	Fuel profile and EV standardization.
Engine_CC	ICE engine-displacement analysis.
Emission_Norm	BS3/BS4/BS6 profile for applicable vehicles.
Vehicle_Age / Vehicle_Age_Group	Fleet-age analysis.

4. End-to-End Analytical Workflow

The project follows a three-layer workflow. Python was used for data preparation and exploratory analysis, PostgreSQL was used for structured SQL analysis and storage, and Power BI was used for interactive modeling and communication.

Stage	Tool	Main Purpose
1. Data preparation	Python + Pandas	Cleaning, standardization, validation, feature creation and exploratory analysis.
2. Database analysis	PostgreSQL + SQL	Store the cleaned dataset and perform aggregations, rankings, filters and cross-tab style analysis.
3. Dashboard modeling	Power BI + DAX	Create dynamic measures, slicers, interactive visuals and presentation-ready insights.

5. Python / Pandas Data Preparation

5.1 Why Pandas was used

Pandas was used because it is well suited to tabular data cleaning, transformation, validation and exploratory analysis. Cleaning the dataset before database loading ensured that the SQL and Power BI layers were based on a consistent analytical table.

5.2 Duplicate removal

```
df = df.drop_duplicates()
```

Duplicate rows were removed so that the same registration record would not be counted more than once. This is particularly important for registration counts, state rankings and manufacturer totals.

5.3 Date conversion and temporal features

```
df['Registration_Date'] = pd.to_datetime(
    df['Registration_Date'],
    format='%d-%m-%Y',
    errors='coerce'
)

df['Registration_Month'] = df['Registration_Date'].dt.month
df['Registration_Day'] = df['Registration_Date'].dt.day
df['Registration_DayOfWeek'] = df['Registration_Date'].dt.day_name()
```

The registration date was converted to a true datetime type. Invalid values are coerced to NaT rather than causing the entire transformation to fail. Month, day and weekday features were derived for temporal analysis.

During validation, ISO-style dates such as 2019-12-13 were identified as incompatible with the original day-month-year format; this reinforced the need to inspect and standardize the date field before analysis.

5.4 State and RTO standardization

A cleaned state field was used to avoid inconsistent state names during aggregation. RTO office strings were also examined to extract a two-letter state prefix where the RTO format supported it.

```
df['RTO_State_Code'] = df['RTO_Office'].str.extract(
    r'\(([A-Z]{2})-\d+\)'
)

print(df['RTO_State_Code'].value_counts())
```

The resulting RTO state-code distribution provided an additional validation dimension and helped identify the geographic composition of the sample.

5.5 Fuel-type cleaning and EV standardization

A cleaned fuel field was created so that vehicles that were clearly pure-electric could be consistently identified. In the working transformation, pure EV manufacturers such as Ola Electric and Ather were used to correct the fuel classification where the original fuel field did not adequately represent the vehicle.

```
pure_ev_brands = ['Ola Electric', 'Ather']

df['Fuel_Type_Clean'] = df['Fuel_Type']

df.loc[
    df['Manufacturer_Brand'].isin(pure_ev_brands),
    'Fuel_Type_Clean'
] = 'EV'

df.loc[
    df['Fuel_Type_Clean'] == 'EV',
    'Engine_CC'
] = 0
```

The purpose of this transformation was to create a consistent EV category and prevent pure EVs from being treated as conventional ICE vehicles in engine-displacement analysis.

5.6 Emission-norm treatment for EVs

A key data-quality correction was made after reviewing the relationship between fuel type and emission norm. Pure battery-electric vehicles do not have conventional tailpipe Bharat Stage classifications such as BS3, BS4 or BS6. Therefore, EV records should not be forced into one of these categories.

```
df.loc[
    df['Fuel_Type_Clean'] == 'EV',
    'Emission_Norm'
] = 'Not Applicable'
```

Why this matters

Without this correction, EVs would contaminate the BS3/BS4/BS6 analysis and could lead to an incorrect interpretation of the transition between conventional tailpipe emission standards.

5.7 Preparation for Vehicle Age and Engine-Size Analysis

Because the working dataset runs through 2024, vehicle age was derived relative to 2024. In Power BI, this was subsequently grouped into presentation-friendly age bands.

Vehicle_Age = 2024 - Registration_Year (used as the basis for Power BI Vehicle_Age_Group)

In Power BI, Vehicle_Age_Group was created with 0–2 Years, 3–5 Years, 6–10 Years and 10+ Years.

Engine_Size_Group was created with <1000 CC, 1000–1499 CC, 1500–1999 CC and 2000+ CC, with EVs kept separate from the ICE engine-size analysis.

5.8 Data-readiness validation

```
target_cols = [
    'Emission_Norm',
    'Vehicle_Category',
    'Registration_Year',
    'Fuel_Type',
    'Fuel_Type_Clean'
]

print(df[target_cols].info())
print(df[target_cols].isnull().sum())

for col in target_cols:
    print(col)
    print(df[col].value_counts(dropna=False))
```

The readiness check was used to verify data types, missing values and unique-value distributions before moving into the SQL and Power BI stages.

5.9 State-focused analysis

```
top_12_states = (
    df['State_Clean']
    .value_counts()
    .head(12)
    .index
    .tolist()
)

df_top12 = df[
    df['State_Clean'].isin(top_12_states)
]

fuel_by_state = pd.crosstab(
    df_top12['State_Clean'],
    df_top12['Fuel_Type_Clean']
)

fuel_by_state['Total'] = fuel_by_state.sum(axis=1)
```

The top-12-state subset was used to compare fuel-type composition across the states with the highest registration volumes. This makes state-level comparisons easier to interpret than displaying every state simultaneously.

6. PostgreSQL / SQL Analysis

After cleaning and validating the data in Pandas, the DataFrame was loaded into PostgreSQL. This created a structured database layer named around the working table `vehicle\_details`, allowing SQL queries to be used independently of the Python notebook.

6.1 Database connection

The PostgreSQL database was accessed from the Jupyter environment using psycopg2/SQLAlchemy-compatible connection logic. The cleaned Pandas DataFrame was then written to the database.

```
df.to_sql(  
    'vehicle_details',  
    engine,  
    if_exists='replace',  
    index=False  
)
```

Using `if\_exists='replace'` was useful during development because the table could be recreated from the current cleaned DataFrame. For production workflows, a controlled append/upsert strategy would be preferable to avoid accidental replacement of an established table.

6.2 Examples of SQL analyses

Analysis	Purpose
Total registered vehicles	Measure the size of the working registration sample.
Vehicles by registration year	Identify growth/decline patterns over time.
Vehicles by state	Identify the leading states by registration volume.
Vehicles by RTO office	Identify high-volume RTOs.
Fuel-type distribution	Understand the composition of the vehicle fleet.
Emission-norm distribution	Quantify BS3/BS4/BS6 for applicable vehicles.
Manufacturer ranking	Identify the brands with the largest number of registrations.
Vehicle-category distribution	Compare 2W, 4W and other categories.
Fuel × state	Compare fuel composition geographically.
Emission norm × fuel	Understand the emission-standard profile of conventional fuel segments.

```
SELECT rto_office,  
       SUM(vehicle_count)  
FROM vehicle_details  
GROUP BY rto_office;
```

Where a dataset contains an explicit vehicle-count field, aggregation should use the correct count measure rather than counting rows blindly. In the final Power BI model, distinct registration numbers were used for the registered-vehicle KPI.

6.3 Detailed SQL Analysis Methodology

The SQL stage was not used only to store the cleaned Pandas DataFrame. PostgreSQL provided a second analytical layer in which the cleaned vehicle data could be queried using relational operations. This made it possible to reproduce important findings independently of the Python notebook and to demonstrate practical SQL skills such as SELECT, WHERE, GROUP BY, ORDER BY, aggregate functions, CASE expressions, DISTINCT, conditional aggregation, subqueries and window functions.

The general SQL workflow was: identify the business question → select the relevant fields → filter invalid or irrelevant records → group the records at the required analytical level → calculate counts or totals → sort/rank the result → validate the result against Pandas and Power BI.

6.4 Basic Data Profiling in SQL

The first SQL checks were designed to understand the size and structure of the database table. These checks are important because they confirm that the table loaded into PostgreSQL contains the expected number of records and that the major analytical fields are populated.

```
-- Total number of rows
SELECT COUNT(*) AS total_rows
FROM vehicle_details;

-- Distinct registered vehicles
SELECT COUNT(DISTINCT registration_number) AS registered_vehicles
FROM vehicle_details;

-- Check distinct fuel types
SELECT fuel_type_clean, COUNT(*) AS vehicle_count
FROM vehicle_details
GROUP BY fuel_type_clean
ORDER BY vehicle_count DESC;
```

The distinction between COUNT(*) and COUNT(DISTINCT registration_number) is important. COUNT(*) counts records, while COUNT(DISTINCT registration_number) counts unique registered vehicles. The latter was used as the conceptual basis for the Power BI Registered Vehicles KPI.

6.5 Registration Trend Analysis

Registration-year analysis was used to identify how the volume of registrations changed over time. Grouping by Registration_Year converts row-level registration records into an annual time series.

```
SELECT
    registration_year,
    COUNT(DISTINCT registration_number) AS registered_vehicles
FROM vehicle_details
GROUP BY registration_year
ORDER BY registration_year;
```

This query supports the Power BI line chart showing registrations by year. SQL is useful here because it produces a compact year-level summary that can be independently checked against the dashboard.

6.6 State-wise Vehicle Registration Analysis

State analysis was performed to identify the geographic concentration of the vehicle fleet. The cleaned State_Clean field was preferred over inconsistent raw state labels so that the same state would not appear under multiple spellings.

```
SELECT
    state_clean,
    COUNT(DISTINCT registration_number) AS registered_vehicles
FROM vehicle_details
GROUP BY state_clean
ORDER BY registered_vehicles DESC;
```

This query answers: Which states have the largest number of registered vehicles in the working dataset? The result was used for the leading-states visual and for identifying states that may deserve deeper emission-impact investigation.

6.7 Top 12 State Analysis

A Top-12 state analysis was used to reduce visual clutter and focus comparative analysis on the states with the highest registration volumes. In SQL, this can be implemented with LIMIT or a ranking window function.

```
SELECT
    state_clean,
    COUNT(DISTINCT registration_number) AS registered_vehicles
FROM vehicle_details
GROUP BY state_clean
ORDER BY registered_vehicles DESC
LIMIT 12;
```

The LIMIT approach is straightforward for a final Top-12 list. A window-function approach is more flexible when the rank itself needs to be retained for later analysis.

```
SELECT *
FROM (
    SELECT
        state_clean,
        COUNT(DISTINCT registration_number) AS registered_vehicles,
        RANK() OVER (
            ORDER BY COUNT(DISTINCT registration_number) DESC
        ) AS state_rank
    FROM vehicle_details
    GROUP BY state_clean
) ranked_states
WHERE state_rank <= 12
ORDER BY state_rank;
```

6.8 Fuel-Type Distribution

Fuel analysis was used to understand the composition of the fleet and to distinguish conventional fuel segments from EVs. The cleaned Fuel_Type_Clean field was used so that corrected EV classifications were included consistently.

```
SELECT
    fuel_type_clean,
    COUNT(DISTINCT registration_number) AS registered_vehicles
FROM vehicle_details
GROUP BY fuel_type_clean
ORDER BY registered_vehicles DESC;
```

This result supports the fuel-distribution visual on Page 1 and provides a baseline for subsequent fuel-specific emission analysis.

6.9 Vehicle Category Analysis

Vehicle-category analysis compares major categories such as two-wheelers and four-wheelers. This is useful because vehicle category influences likely fuel consumption, usage patterns and the type of emission assessment required in future work.

```
SELECT
    vehicle_category,
    COUNT(DISTINCT registration_number) AS registered_vehicles
```



```
FROM vehicle_details
GROUP BY vehicle_category
ORDER BY registered_vehicles DESC;
```

6.10 Manufacturer Analysis

Manufacturer-level aggregation was used to identify the brands with the highest number of registrations. This helps describe market composition and can later support manufacturer/model-specific analysis.

```
SELECT
    manufacturer_brand,
    COUNT(DISTINCT registration_number) AS registered_vehicles
FROM vehicle_details
GROUP BY manufacturer_brand
ORDER BY registered_vehicles DESC
LIMIT 10;
```

The Top-10 result was used in the Power BI manufacturer visual. LIMIT was used because the objective was to highlight the most important manufacturers rather than display every brand.

6.11 Emission-Norm Analysis

Emission-norm analysis is central to the project's environmental objective. After correcting EV records, BS3, BS4 and BS6 are treated as applicable classifications for the relevant vehicle population, while EV records are treated as Not Applicable.

```
SELECT
    emission_norm,
    COUNT(DISTINCT registration_number) AS registered_vehicles
FROM vehicle_details
GROUP BY emission_norm
ORDER BY registered_vehicles DESC;
```

This query provides the overall emission-norm profile. It should be interpreted as a distribution of regulatory emission-standard classifications, not as a direct measurement of CO₂ emissions.

6.12 Correct BS3/BS4/BS6 Analysis Excluding EVs

A particularly important SQL validation is to analyze Bharat Stage norms only for vehicles where the classification is applicable. This prevents EVs from being incorrectly counted as BS3, BS4 or BS6.

```
SELECT
    emission_norm,
    COUNT(DISTINCT registration_number) AS registered_vehicles
FROM vehicle_details
WHERE fuel_type_clean <> 'EV'
    AND emission_norm IN ('BS3', 'BS4', 'BS6')
GROUP BY emission_norm
ORDER BY
    CASE emission_norm
        WHEN 'BS3' THEN 1
        WHEN 'BS4' THEN 2
        WHEN 'BS6' THEN 3
    END;
```

For the current working sample, the corrected dashboard shows 4 BS3, 451 BS4 and 2,277 BS6 vehicles, while 968 EVs are kept outside the Bharat Stage classification. This is an important data-quality and domain-logic decision.

6.13 Fuel Type × Emission Norm Cross-Analysis

A cross-analysis between fuel type and emission norm was used to determine how conventional fuel segments are distributed across BS3, BS4 and BS6. This is more informative than looking at either variable alone.

```
SELECT
    fuel_type_clean,
    emission_norm,
    COUNT(DISTINCT registration_number) AS registered_vehicles
FROM vehicle_details
WHERE fuel_type_clean <> 'EV'
    AND emission_norm IN ('BS3', 'BS4', 'BS6')
GROUP BY fuel_type_clean, emission_norm
ORDER BY fuel_type_clean, emission_norm;
```

The resulting two-dimensional summary was represented as a matrix in Power BI. It allows a viewer to compare Petrol, Diesel, CNG and Hybrid segments across applicable emission standards.

6.14 State × Fuel Analysis

State-by-fuel analysis identifies whether the fuel composition changes geographically. This is useful for future emission estimation because different fuel mixes imply different emission-factor requirements.

```
SELECT
    state_clean,
    fuel_type_clean,
    COUNT(DISTINCT registration_number) AS registered_vehicles
FROM vehicle_details
GROUP BY state_clean, fuel_type_clean
ORDER BY state_clean, registered_vehicles DESC;
```

For presentation, the analysis was restricted to the leading states where necessary, matching the Top-12 state analysis performed in Pandas.

6.15 RTO-level Analysis

RTO analysis was used to move from state-level geography to administrative registration-office geography. The RTO office field can reveal high-volume registration centres and can support future regional sampling.

```
SELECT
    rto_office,
    COUNT(DISTINCT registration_number) AS registered_vehicles
FROM vehicle_details
GROUP BY rto_office
ORDER BY registered_vehicles DESC
LIMIT 20;
```

Where a separate vehicle_count field exists in a source table, SUM(vehicle_count) may be more appropriate than COUNT(DISTINCT registration_number). The correct aggregation depends on the grain of the source table.

6.16 Vehicle Age Analysis in SQL

Vehicle age can be derived from registration year and then grouped into meaningful bands. Doing this in SQL is useful when the database needs to supply an already summarized age profile.

```
SELECT
    CASE
```

```

        WHEN 2024 - registration_year <= 2 THEN '0-2 Years'
        WHEN 2024 - registration_year <= 5 THEN '3-5 Years'
        WHEN 2024 - registration_year <= 10 THEN '6-10 Years'
        ELSE '10+ Years'
    END AS vehicle_age_group,
    COUNT(DISTINCT registration_number) AS registered_vehicles
FROM vehicle_details
GROUP BY vehicle_age_group
ORDER BY vehicle_age_group;

```

The same logic was implemented in Power BI as a calculated column for interactive visualization. The important principle is that the group is based on registration age, not on an assumed measured operating age.

6.17 ICE Engine-Displacement Analysis

Engine displacement was analyzed separately from EVs. Since EV records were represented with Engine_CC = 0 in the working model, including them in a conventional average would artificially reduce the average ICE engine displacement.

```

SELECT
    AVG(engine_cc) AS average_ice_engine_cc
FROM vehicle_details
WHERE fuel_type_clean <> 'EV'
    AND engine_cc IS NOT NULL;

```

This SQL logic is equivalent to the Power BI Average ICE Engine CC measure. It is a better metric for an ICE-engine profile than an overall average that includes zero-valued EV records.

6.18 Identifying Older and Legacy Emission Segments

SQL can also be used to identify segments that may deserve priority in a future activity-based emission study. These are not direct CO₂ measurements; they are screening indicators.

```

SELECT
    COUNT(DISTINCT registration_number) AS older_vehicles
FROM vehicle_details
WHERE 2024 - registration_year >= 10
    AND fuel_type_clean <> 'EV';

SELECT
    emission_norm,
    COUNT(DISTINCT registration_number) AS registered_vehicles
FROM vehicle_details
WHERE emission_norm IN ('BS3', 'BS4')
    AND fuel_type_clean <> 'EV'
GROUP BY emission_norm
ORDER BY emission_norm;

```

The first query identifies older ICE vehicles, while the second isolates legacy BS3/BS4 segments. These groups can later be combined with actual activity data to estimate emissions or prioritize field studies.

6.19 Data-Quality Validation with SQL

SQL was also useful as an independent validation layer. The cleaned table can be checked for missing values, unexpected categories and inconsistent EV/emission-norm combinations.

```

-- Missing registration years
SELECT COUNT(*) AS missing_registration_year
FROM vehicle_details
WHERE registration_year IS NULL;

```

```

-- EVs incorrectly carrying a BS classification
SELECT
    emission_norm,
    COUNT(*) AS records
FROM vehicle_details
WHERE fuel_type_clean = 'EV'
    AND emission_norm IN ('BS3', 'BS4', 'BS6')
GROUP BY emission_norm;

-- Unexpected fuel categories
SELECT DISTINCT fuel_type_clean
FROM vehicle_details
ORDER BY fuel_type_clean;

```

The EV/emission-norm query should return zero records after the EV correction. This is an example of using SQL not just for reporting, but also for data-quality assurance.

6.20 SQL Aggregations Used in the Project

6.21 How SQL Results Connected to Power BI

The SQL layer and Power BI layer served different purposes. SQL was primarily used to validate, summarize and interrogate the database. Power BI then used the cleaned table and DAX measures to provide interactive analysis. This separation is useful because a SQL query can answer a fixed analytical question, whereas a DAX measure can recalculate the same concept dynamically when a user changes Year, State, Fuel Type or Emission Norm slicers.

- SQL: 'How many vehicles are registered in each state?' → fixed grouped result.
- Power BI: State slicer + Registered Vehicles measure → the KPI recalculates for the selected state.
- SQL: 'How many BS6 vehicles are present?' → fixed count for the queried dataset.
- Power BI: BS6 Vehicles measure → dynamically recalculates after Year/State/Fuel filters.
- SQL: 'What is the average ICE engine CC?' → fixed aggregate for the selected query scope.
- Power BI: Average ICE Engine CC → dynamically recalculates according to the current dashboard filter context.

6.22 SQL Analysis Summary

Overall, SQL transformed the cleaned dataset from a collection of records into reproducible analytical summaries. The SQL work covered fleet size, registration trends, state and RTO concentration, fuel composition, vehicle categories, manufacturer rankings, emission norms, fuel-by-emission-norm relationships, vehicle age, engine displacement and data-quality validation. These results were then carried into Power BI to create the final interactive dashboard.

7. Key Analytical Findings

7.1 Overall fleet size and composition

The working dataset contains 3,700 registration records. The dashboard reports 1,648 two-wheelers and 1,160 four-wheelers. Additional categories such as 3W, LCV, HCV and OTH are also present. This indicates that the dataset is not limited to passenger cars; it represents a broader vehicle fleet.

7.2 EV representation

After fuel-type cleaning, 968 vehicles are classified as EVs. EVs are presented separately from the BS3/BS4/BS6 analysis because conventional Bharat Stage tailpipe standards are not applicable to pure battery-electric vehicles.

7.3 Emission-norm profile

Classification	Vehicles	Interpretation
BS3	4	Small legacy segment in the working sample.
BS4	451	Older applicable tailpipe-standard segment.
BS6	2,277	Dominant applicable Bharat Stage segment.
Not Applicable / EV	968	Pure EV segment; excluded from BS3/BS4/BS6 comparison.
Total	3,700	Complete working sample.

The emission-norm distribution indicates a strong presence of BS6 vehicles among records with an applicable Bharat Stage classification. The small BS3 count and larger BS4 count are useful for identifying legacy segments for further assessment.

7.4 Fuel and emission-norm relationship

The current matrix for applicable BS classifications contains CNG, Diesel, Hybrid and Petrol records. The applicable-norm totals are 4 BS3, 451 BS4 and 2,277 BS6, for 2,732 records. EVs are not included in that matrix after the correction described above.

7.5 Registration trend

The registration trend rises from 2018 through 2023 in the working sample, with 2023 representing the highest point in the displayed series. Registrations then decline in 2024. This should be interpreted carefully because a lower final-year value can be affected by data coverage, reporting completeness or the scope of the source extract.

7.6 Geographic concentration

The leading states visible in the dashboard include Karnataka, Kerala, Madhya Pradesh, Tamil Nadu, Delhi and Rajasthan. The state analysis is useful because vehicle emissions and fleet-transition policies are implemented geographically; state and RTO concentration can therefore guide future activity-data collection and emission-estimation work.

7.7 Manufacturer concentration

The top manufacturers in the dashboard include Honda, Mahindra, TVS, Tata Motors, Bajaj and Suzuki. Manufacturer concentration provides useful context for fleet composition, technology transition and future vehicle-segment prioritization.

7.8 Vehicle age and engine displacement

Vehicle age was derived from registration year relative to 2024 and grouped into age bands for Power BI. Engine displacement was analyzed only for ICE vehicles, while EVs were separated from the engine-CC

analysis. This prevents zero-valued EV engine fields from artificially lowering the average ICE engine displacement.

8. Power BI Dashboard Development

Power BI was used as the presentation and decision-support layer. The dashboard was intentionally designed as a light-theme, interactive report rather than a static collection of charts. Slicers and DAX measures allow the same visuals to respond dynamically to user selections.

8.1 Core dynamic measures

```
Registered Vehicles =  
DISTINCTCOUNT(  
    vehicle_details[Registration_Number]  
)  
  
2W Vehicles =  
CALCULATE(  
    [Registered Vehicles],  
    vehicle_details[Vehicle_Category] = "2W"  
)  
  
4W Vehicles =  
CALCULATE(  
    [Registered Vehicles],  
    vehicle_details[Vehicle_Category] = "4W"  
)  
  
EV Vehicles =  
CALCULATE(  
    [Registered Vehicles],  
    vehicle_details[Fuel_Type_Clean] = "EV"  
)  
  
BS6 Vehicles =  
CALCULATE(  
    [Registered Vehicles],  
    vehicle_details[Emission_Norm] = "BS6"  
)  
  
BS4 Vehicles =  
CALCULATE(  
    [Registered Vehicles],  
    vehicle_details[Emission_Norm] = "BS4"  
)  
  
Average ICE Engine CC =  
CALCULATE(  
    AVERAGE(vehicle_details[Engine_CC]),  
    vehicle_details[Fuel_Type_Clean] <> "EV"  
)
```

The distinction between calculated columns and measures was important. Columns such as vehicle-age groups are row-level transformations, whereas KPIs such as registered vehicles and average ICE engine CC are measures that must respond dynamically to slicers and visual interactions.

8.2 Power BI — Additional Derived Columns

Two additional analytical columns were created in Power BI after the cleaned dataset was loaded. These columns were created for dashboard analysis and presentation; they were not part of the original Pandas data-cleaning process.

1. Vehicle_Age_Group

Created from Vehicle_Age and grouped into 0–2 Years, 3–5 Years, 6–10 Years and 10+ Years. The grouping makes the age profile easier to interpret in the dashboard.

2. Engine_Size_Group

Created from Engine_CC and grouped into <1000 CC, 1000–1499 CC, 1500–1999 CC and 2000+ CC. The engine-size analysis is focused on ICE vehicles, with EVs kept separate.

8.3 Page 1 — Fleet Overview

Purpose: This page provides an overall view of the registered vehicle fleet and its composition.

Slicers

- Year
- State
- Fuel Type

KPI Cards

- Registered Vehicles
- 2W Vehicles
- 4W Vehicles
- EV Vehicles
- Average Vehicle Age

Dashboard Visuals

- Registration by Year — shows the registration trend from 2018 to 2024.
- Vehicle Registrations by Category — compares 2W, 4W, 3W, LCV, HCV and OTH.
- Top Manufacturers — displays the leading vehicle manufacturers.
- Leading States by Registered Vehicles — highlights the states with the highest registration counts.

8.4 Page 2 — Emission Profile

- Slicers: Year, State, Fuel Type and Emission Norm.
- KPI cards: Registered Vehicles, BS6 Vehicles, BS4 Vehicles, EV Vehicles and Average ICE Engine CC.
- Applicable Emission Norm by Year — ICE Vehicles.
- Fuel Type × Applicable Emission Norm matrix.
- Vehicle Age Profile.
- Vehicle Category × Emission Norm chart.
- Engine Size Distribution.

Purpose: Page 2 focuses on the emission profile of the fleet. It combines applicable Bharat Stage emission norms with fuel type, vehicle age, vehicle category and ICE engine displacement, while keeping pure EVs separate from conventional BS3/BS4/BS6 and engine-CC analysis.

The completed Power BI dashboard contains two pages only: Page 1 — Fleet Overview and Page 2 — Emission Profile. The third-page concept is a future enhancement and was not included in the completed dashboard.

8. Dynamic and Interactive Features

- Year slicer changes KPIs and all time-dependent visuals.
- State slicer allows geographic drill-down.
- Fuel Type and Emission Norm slicers allow focused segment analysis.
- Visual cross-filtering allows a selection in one chart to affect other visuals.
- Page navigation provides a structured flow between the two completed dashboard pages: Fleet Overview and Emission Profile.
- DAX measures ensure KPI values are recalculated rather than hard-coded.
- Slicer synchronization can be used across pages so that the same analytical context is maintained throughout the report.

10. Data Quality and Validation Decisions

Issue / Decision	Action Taken	Reason
Duplicate records	Removed duplicate rows	Prevent double counting.
Date formatting	Converted registration date to datetime; invalid formats handled with coercion/validation	Enable reliable temporal analysis.
State inconsistencies	Used State_Clean	Consistent geographic aggregation.
RTO coding	Extracted RTO state prefix where available	Validate and analyze RTO geography.
EV fuel classification	Created Fuel_Type_Clean	Separate pure EVs consistently.
EV engine CC	Set Engine_CC to 0 for EV records	Represent absence of conventional engine displacement in the working model.
EV emission norm	Set to Not Applicable	Prevent incorrect BS3/BS4/BS6 classification.
Vehicle age	Derived from 2024 minus registration year	Measure fleet age.
Engine displacement	Grouped for visualization and analyzed as ICE-only	Avoid mixing EVs with ICE engine sizes.
Readiness check	Checked info, nulls and value counts	Validate analytical fields before dashboarding.

10. Limitations

- The analysis represents the supplied working sample and should not automatically be interpreted as the complete national vehicle population.
- Registration counts do not equal vehicle usage or annual kilometres travelled.
- A registered vehicle does not provide its actual fuel consumption or real-world emissions.
- Emission norms describe regulatory standards/classification; they are not direct measurements of CO₂ emissions.
- The EV classification depends on the cleaning rules and source fields used in the dataset.

- The 2024 registration count should be interpreted in the context of the coverage/completeness of the supplied extract.
- Carbon-credit quantities cannot be derived reliably without activity data, emission factors and an approved accounting methodology.

11. Future Scope

10. Integrate annual vehicle-kilometres travelled or other activity data.
11. Add fuel-consumption or fuel-economy information by vehicle category and fuel type.
12. Apply validated emission factors to estimate CO₂ and CO₂e by vehicle segment.
13. Build baseline-versus-project scenarios for fleet modernization, cleaner fuels and electrification.
14. Incorporate vehicle survival/scrappage assumptions to model fleet turnover.
15. Add RTO-level and state-level emission estimates once activity data is available.
16. Evaluate emission-reduction projects under a suitable carbon-credit accounting methodology rather than assigning credits directly from registrations.
17. Automate the pipeline so that new Vahan extracts can refresh PostgreSQL and Power BI with minimal manual work.
18. Extend the analysis with vehicle activity data, fuel or energy consumption and validated emission factors to estimate CO₂e emissions and investigate emission-reduction opportunities and the potential applicability of carbon-credit mechanisms.

12. Conclusion

The project demonstrates a complete data analytics pipeline for vehicle-registration data. Pandas was used to clean, validate and enrich the raw data; PostgreSQL provided a structured SQL environment for querying and aggregation; and Power BI transformed the results into an interactive decision-support dashboard.

The most important analytical improvement was separating pure EVs from conventional Bharat Stage emission-norm analysis. This prevents EVs from being incorrectly assigned BS3, BS4 or BS6 classifications and allows the dashboard to present a more defensible distinction between EV fleet composition and ICE tailpipe-emission standards.

The completed two-page dashboard therefore provides a strong baseline for vehicle-fleet and emission-profile research. It identifies where vehicles are registered, which categories and manufacturers dominate, how the fleet has evolved over time, how applicable emission norms are distributed, and which segments may deserve further emission assessment. The next step toward true carbon-emission and carbon-credit analysis is not to infer credits from registrations alone, but to integrate vehicle activity, fuel/energy consumption and validated emission factors into the analytical model.

Appendix A — Core Dashboard Measures

```
Registered Vehicles =
DISTINCTCOUNT(vehicle_details[Registration_Number])
```

```
2W Vehicles =
CALCULATE(
    [Registered Vehicles],
```

```

        vehicle_details[Vehicle_Category] = "2W"
    )

4W Vehicles =
CALCULATE(
    [Registered Vehicles],
    vehicle_details[Vehicle_Category] = "4W"
)

EV Vehicles =
CALCULATE(
    [Registered Vehicles],
    vehicle_details[Fuel_Type_Clean] = "EV"
)

EV Share =
DIVIDE(
    [EV Vehicles],
    [Registered Vehicles],
    0
)

BS6 Vehicles =
CALCULATE(
    [Registered Vehicles],
    vehicle_details[Emission_Norm] = "BS6"
)

BS4 Vehicles =
CALCULATE(
    [Registered Vehicles],
    vehicle_details[Emission_Norm] = "BS4"
)

Average Vehicle Age =
AVERAGE(vehicle_details[Vehicle_Age])

Average ICE Engine CC =
CALCULATE(
    AVERAGE(vehicle_details[Engine_CC]),
    vehicle_details[Fuel_Type_Clean] <> "EV"
)

```

Appendix B — Suggested Professional Presentation Statement

“This project analyzes Vahan vehicle-registration data through a complete Python–SQL–Power BI pipeline. The data was cleaned and standardized using Pandas, loaded into PostgreSQL for structured SQL analysis, and modeled in Power BI to create dynamic fleet and emission-profile dashboards. The analysis focuses on vehicle composition, registration trends, geographic concentration, fuel type, vehicle age, engine displacement and applicable Bharat Stage emission norms. EVs are treated separately because conventional BS3/BS4/BS6 tailpipe classifications are not applicable to pure battery-electric vehicles. The resulting dashboard establishes a baseline for future activity-based CO₂e estimation and emission-reduction research.”

SQL concept	How it was used	Analytical value
COUNT / COUNT(DISTINCT)	Vehicle and registration counts	Measure fleet size without double

		counting.
SUM	Aggregate explicit vehicle-count fields where applicable	Calculate total vehicle volumes.
AVG	Average ICE engine displacement	Characterize conventional engine profile.
GROUP BY	Year, state, fuel, category, manufacturer, emission norm	Convert row-level data into analytical summaries.
ORDER BY	Rank states, manufacturers, RTOs and years	Identify leading/high-volume segments.
WHERE	Filter EVs, BS3/BS4/BS6 and date/segment conditions	Create analytically valid subsets.
CASE	Create age bands and custom classifications	Turn continuous values into interpretable groups.
DISTINCT	Unique registration counts	Avoid duplicate-record inflation.
RANK / window functions	Top-state ranking	Retain rank information for downstream analysis.
Subqueries	Ranked Top-12/Top-N analyses	Enable multi-stage analytical filtering.

Report Prepared By

Anurag Patil

B.Tech — Computer Science & Engineering (Artificial Intelligence & Machine Learning)

Project

Vahan Vehicle Registration Data Analysis

Technology Stack

- Python: Pandas, NumPy, Matplotlib
- Database: PostgreSQL / SQL
- Visualization & BI: Microsoft Power BI

Data Analysis Report — 2026